

Análise comparativa de algoritmos com uso de paralelismo

Autor 1: Guilherme Braga

Autor 2: Miguel Tacchi

Resumo

Este relatório apresenta um estudo comparativo entre algoritmos de contagem de palavras implementados em ambientes seriais e paralelos utilizando Java. O estudo implementou e comparou três abordagens: uma versão serial na CPU, uma versão paralela com uso de threads e uma versão paralela utilizando GPU com OpenCL. Os experimentos foram realizados utilizando arquivos de texto de tamanhos variados, e os tempos de execução foram registrados em arquivos CSV para posterior análise visual por meio de gráficos. Os resultados demonstram diferenças significativas de desempenho entre os métodos, evidenciando o impacto do paralelismo e do hardware disponível no tempo de processamento.

Introdução

A busca por eficiência computacional é um dos pilares da computação moderna, especialmente em aplicações que manipulam grandes massas de dados. Em tais aplicações, explorar os recursos de hardware disponíveis — como múltiplos núcleos de CPU ou unidades de processamento gráfico (GPU) — pode proporcionar ganhos substanciais de desempenho. O trabalho investiga três métodos de contagem de ocorrências de uma palavra em arquivos de texto:

- **SerialCPU** – Implementação serial utilizando um único *thread* com iteração simples.
- **ParallelCPU** – Implementação paralela utilizando *Thread Pools* do Java, dividindo o texto em partes e processando-as simultaneamente.
- **ParallelGPU** – Implementação com OpenCL (utilizando a biblioteca JOCL), onde a contagem é executada de forma massivamente paralela na GPU.

O objetivo principal é comparar o desempenho dessas abordagens em diferentes cenários, avaliando como o tamanho do conjunto de dados e o nível de paralelismo impactam os resultados.

Metodologia

A metodologia do projeto concentrou-se na implementação e comparação rigorosa de três abordagens de contagem de palavras: Serial na CPU, Paralela na CPU (usando *Thread Pools* com variação de núcleos) e paralela na GPU (via *kernels* OpenCL). O processo envolveu a criação de um *framework* que permite:

Execução Repetida: Cada teste foi rodado no mínimo três vezes em arquivos de texto de diferentes tamanhos para garantir a consistência estatística.

Registro de Dados: Os resultados de tempo (*tempo_ms*) e ocorrências foram exportados para arquivos CSV.

Análise: O CSV gerado permitiu a visualização gráfica do desempenho, focando na escalabilidade da CPU paralela e nas limitações de *overhead* do uso da GPU em tarefas de processamento de texto.

Resultados e Discussão

Os experimentos realizados com as obras literárias *Dracula*, *Don Quixote* e *Moby Dick* revelaram padrões de comportamento consistentes entre as diferentes abordagens computacionais. As Figuras 1, 2 e 3 (referentes aos gráficos de execução) ilustram os tempos de processamento em milissegundos para cada método.

Desempenho do Paralelismo em CPU (Thread Pools): A abordagem **ParallelCPU** demonstrou ser a mais eficiente em todos os cenários testados.

Escalabilidade: Observa-se que, com o aumento do número de threads (de 2 para 10), o tempo de execução tende a diminuir e se estabilizar.

Comparação com Serial: O ganho de desempenho é notável. No caso do texto *Don Quixote* (que apresentou os maiores tempos absolutos, sugerindo ser o maior arquivo), a execução Serial levou cerca de **400ms**, enquanto a execução paralela com 10 threads ficou abaixo de **100ms** — um *speedup* de aproximadamente 4x.

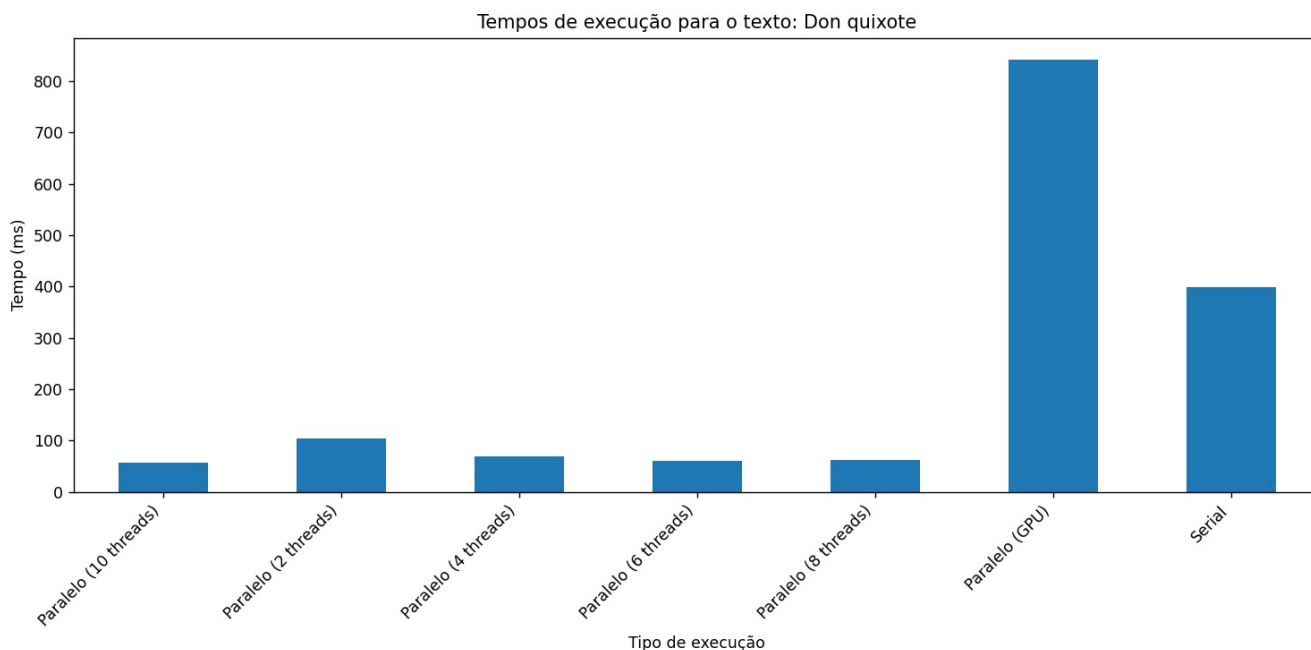
Consistência: Nos textos *Dracula* e *Moby Dick*, as versões paralelas mantiveram-se na faixa de 20ms a 50ms, superando consistentemente a versão serial.

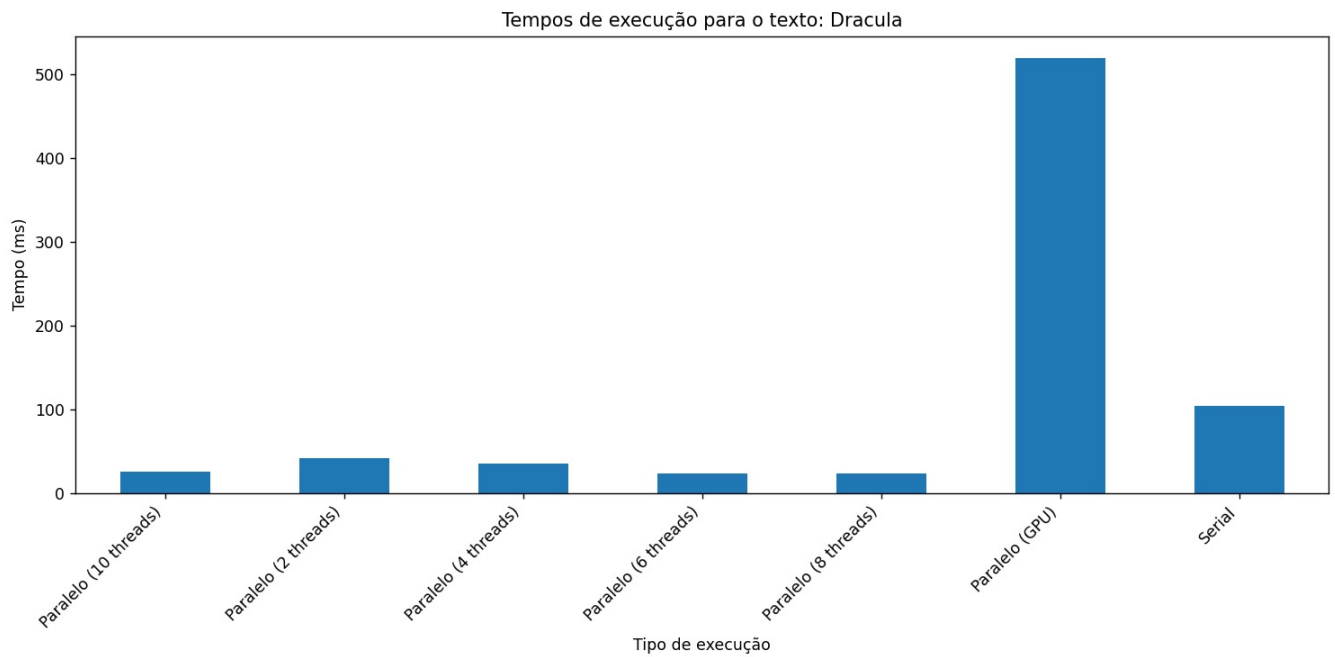
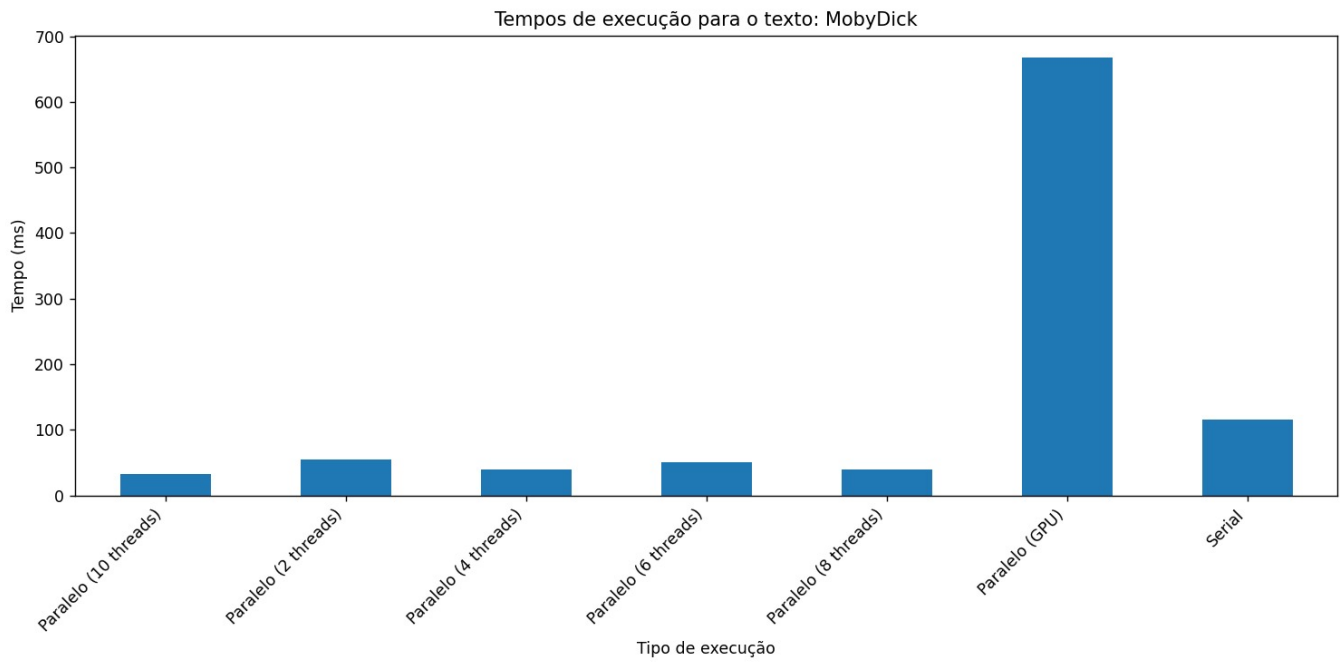
O Gargalo da GPU (JOCL): A análise dos gráficos revela um comportamento contra-intuitivo para a abordagem **ParallelGPU**, ela foi consistentemente a mais lenta, com tempos variando entre **500ms** (*Dracula*) e mais de **800ms** (*Don Quixote*).

Análise de Custo: Isso confirma a hipótese de que o *overhead* (custo adicional) de comunicação entre o Host (CPU/RAM) e o Device (GPU/VRAM) supera o ganho de processamento paralelo para tarefas de contagem de palavras em arquivos deste porte.

Inicialização: O tempo de compilação do Kernel OpenCL e a alocação de buffers consomem mais tempo do que a própria contagem simples realizada pela CPU. A GPU só se justificaria para volumes de dados massivos (na ordem de Gigabytes) ou cálculos matemáticos mais complexos por byte processado.

Abordagem Serial: A execução **SerialCPU** serviu como um *baseline* robusto. Embora mais lenta que a CPU paralela, ela foi significativamente mais rápida que a GPU nestes testes. O gráfico de *Don Quixote* evidencia que, conforme o tamanho do texto aumenta, a diferença entre Serial e ParallelCPU se torna mais acentuada, justificando o uso de *multithreading* em CPUs modernas para processamento de texto.





Conclusão

para a tarefa de contagem de palavras, o paralelismo na CPU (multi-threading) se mostrou a abordagem mais eficiente e rápida, alcançando o melhor *speedup* (ganho de velocidade).

A execução Serial foi uma linha de base estável, porém lenta. Já a abordagem Paralela na GPU (OpenCL) foi, consistentemente, o método mais lento de todos, devido ao alto custo (*overhead*) de transferência de dados entre a memória do sistema (CPU) e a memória da placa de vídeo, custo esse que superou amplamente o benefício do processamento paralelo massivo para esta tarefa simples de processamento de texto.

Anexos

Os códigos completos das três versões (SerialCPU, ParallelCPU e ParallelGPU), além dos kernels OpenCL utilizados, encontram-se no repositório:

<https://github.com/GuilhermeBrga/contador-de-palavras---cmpt-paralela.git>

Referências

Biblioteca jocl-2.0.4.
Amostras fornecidas na atividade
Documentação oficial OpenCL